

Lab Report: UPI Payment System

Demonstrating OOP Concepts, Custom Exceptions, and Interface Implementation in Java

Course:	Object-Oriented Programming (Java)	Package:	observation
Main Class:	UpiPayments.java	Concepts:	Encapsulation, Interfaces, Exception Handling

1. Objective / Aim

To design and implement a digital wallet & payment system in Java using **Object-Oriented Programming (OOP) principles**, simulating real-world payment operations such as wallet initialization, balance management, UPI ID format validation, payment processing, and robust custom exception handling.

2. Problem Statement

Design a **UPI Payment & Wallet Service** for a user. The system must allow users to:

- Create a wallet with details: `userName`, `mobileNumber`, `upiId`, and initial balance.
- Add funds to the wallet with positive amount checks.
- Make payments to recipient UPI IDs with strict format validation.
- Prevent transactions if funds are insufficient or transaction amounts are invalid (≤ 0).
- Handle failed operations cleanly using **Custom Exceptions** and `try-catch-finally` blocks.
- Log balance updates after every transaction attempt regardless of success or failure.

3. Class and Interface Architecture

Component	Type	Description
PaymentService	Interface	Defines abstract contract with <code>pay(...)</code> and <code>checkBalance()</code> .
Wallet	Encapsulated Class	Encapsulates user profile and balance state with accessors/mutators.
UpiPaymentService	Implementation Class	Implements <code>PaymentService</code> ; contains validation and business logic.
InsufficientBalanceException	Custom Exception	Thrown when transaction amount exceeds current available balance.
InvalidUPIException	Custom Exception	Thrown when receiver's UPI ID fails format checks (missing '@' or domain '.').
InvalidAmountException	Custom Exception	Thrown when transfer or deposit amount is zero or negative.



4. Step-by-Step Execution Process

- 1 **Define Custom Exceptions:** Create `InsufficientBalanceException`, `InvalidUPIException`, and `InvalidAmountException` extending `java.lang.Exception`.
- 2 **Define Interface:** Declare the contract in `PaymentService` specifying expected payment and balance checking behaviors.
- 3 **Encapsulate Wallet:** Store user fields privately. Implement safety guard inside constructor (`Math.max(0, initialBalance)`) and explicit methods for adding/deducting money.
- 4 **Implement UPI Payment System:** Associate `Wallet` via dependency injection in `UpiPaymentService`. Implement `pay()` with sequential validations:
 - *UPI Format Validation:* Checks for non-null, presence of @, and a domain dot . following @.
 - *Amount Validation:* Ensures `amount > 0`.
 - *Balance Check:* Verifies available balance prior to deduction.
- 5 **Exception Handling & Transaction Logging:** Implement a driver method using `try-catch-finally` blocks to guarantee that transaction logs and balance checks are performed post-transaction attempt.



5. UML Class Diagram & Architecture

<<interface>> PaymentService

```
+ pay(receiverUpiId: String, amount: double):  
void  
+ checkBalance(): void
```

Custom Exceptions

```
+ InsufficientBalanceException  
+ InvalidUPIException  
+ InvalidAmountException
```

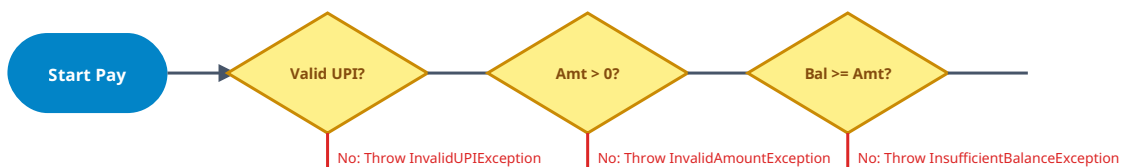
Wallet

```
- userName: String  
- mobileNumber: String  
- upiId: String  
- balance: double  
  
+ Wallet(userName, mobile, upi, initialBal)  
+ getUserName(): String  
+ getBalance(): double  
+ addMoney(amount: double): void  
+ deductMoney(amount: double): void  
+ displayWalletDetails(): void
```

UpiPaymentService (Implements PaymentService)

```
- wallet: Wallet  
  
+ UpiPaymentService(wallet: Wallet)  
+ pay(receiverUpiId: String, amount: double): void throws InvalidUPIException,  
InvalidAmountException, InsufficientBalanceException  
+ checkBalance(): void
```

Transaction Validation & Execution Workflow



6. Java Source Code Implementation

```
package observation;  
  
//=====  
// 1. Custom Exceptions  
//=====  
class InsufficientBalanceException extends Exception {  
    private static final long serialVersionUID = 1L;  
  
    public InsufficientBalanceException(String message) {  
        super(message);  
    }  
}  
  
class InvalidUPIException extends Exception {  
    private static final long serialVersionUID = 1L;  
  
    public InvalidUPIException(String message) {
```

```

        super(message);
    }
}

class InvalidAmountException extends Exception {
    private static final long serialVersionUID = 1L;

    public InvalidAmountException(String message) {
        super(message);
    }
}

//=====
// 2. PaymentService Interface
//=====
interface PaymentService {
    void pay(String receiverUpiId, double amount)
        throws InvalidUPIException, InvalidAmountException, InsufficientBalanceException;
    void checkBalance();
}

//=====
// 3. Wallet Class (Encapsulated)
//=====
class Wallet {
    private String userName;
    private String mobileNumber;
    private String upiId;
    private double balance;

    public Wallet(String userName, String mobileNumber, String upiId, double initialBalance) {
        this.userName = userName;
        this.mobileNumber = mobileNumber;
        this.upiId = upiId;
        this.balance = Math.max(0, initialBalance);
    }

    public String getUserName() { return userName; }
    public String getMobileNumber() { return mobileNumber; }
    public String getUpiId() { return upiId; }
    public double getBalance() { return balance; }

    public void addMoney(double amount) throws InvalidAmountException {
        if (amount <= 0) {
            throw new InvalidAmountException("Amount to add must be greater than zero.");
        }
        this.balance += amount;
        System.out.println("Successfully added ₹" + amount + " to wallet.");
    }

    public void deductMoney(double amount) {
        this.balance -= amount;
    }

    public void displayWalletDetails() {
        System.out.println("
        ----- Wallet Details -----");
        System.out.println("User Name      : " + userName);
        System.out.println("Mobile Number : " + mobileNumber);
        System.out.println("UPI ID       : " + upiId);
        System.out.println("Current Balance: ₹" + balance);
        System.out.println("-----");
    }
}

```

```

//=====
// 4. Payment Implementation
//=====
class UpiPaymentService implements PaymentService {
    private Wallet wallet;

    public UpiPaymentService(Wallet wallet) {
        this.wallet = wallet;
    }

    @Override
    public void pay(String receiverUpiId, double amount)
        throws InvalidUPIException, InvalidAmountException, InsufficientBalanceException {

        // 1. Validate UPI ID format safely
        if (receiverUpiId == null || !receiverUpiId.contains("@")) {
            throw new InvalidUPIException("Invalid UPI ID format: '" + receiverUpiId + "'.");
        }

        int atIndex = receiverUpiId.indexOf("@");
        if (atIndex == receiverUpiId.length() - 1 || !receiverUpiId.substring(atIndex).contains("."))
        {
            throw new InvalidUPIException("Invalid UPI ID format: '" + receiverUpiId + "'.");
        }

        // 2. Validate payment amount
        if (amount <= 0) {
            throw new InvalidAmountException("Payment amount must be greater than zero.");
        }

        // 3. Check for sufficient balance
        if (wallet.getBalance() < amount) {
            throw new InsufficientBalanceException("Insufficient balance! Requested: ₹" + amount +
            ", Available: ₹" + wallet.getBalance());
        }

        // Deduct money if all validations pass
        wallet.deductMoney(amount);
        System.out.println("Payment of ₹" + amount + " to " + receiverUpiId + " was successful!");
    }

    @Override
    public void checkBalance() {
        System.out.println("Available Balance: ₹" + wallet.getBalance());
    }
}

//=====
// 5. Main Execution Class (File Name: UpiPayments.java)
//=====
public class UpiPayments {
    public static void main(String[] args) {
        // Step 1: Create a wallet
        Wallet userWallet = new Wallet("REVANTH REDDY", "9652569999", "krevanthreddy2@upi", 1000.0);
        UpiPaymentService upiSystem = new UpiPaymentService(userWallet);

        userWallet.displayWalletDetails();

        // Step 2: Add money to wallet
        try {
            System.out.println("
Adding ₹500 to wallet...");
            userWallet.addMoney(500.0);
        } catch (InvalidAmountException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}

```

```

    }

    // Step 3: Test valid transaction using try-catch-finally
    executeTransaction(upiSystem, "merchant@okaxis", 300.0);

    // Step 4: Test Invalid UPI ID Exception
    executeTransaction(upiSystem, "invalidUpiFormat", 200.0);

    // Step 5: Test Negative / Zero Amount Exception
    executeTransaction(upiSystem, "friend@okicici", -50.0);

    // Step 6: Test Insufficient Balance Exception
    executeTransaction(upiSystem, "friend@okicici", 5000.0);

    // Final Wallet Details
    userWallet.displayWalletDetails();
}

// Helper method to demonstrate try-catch-finally block
private static void executeTransaction(UpiPaymentService system, String receiverUpi, double
amount) {
    System.out.println("
Attempting transaction to " + receiverUpi + " of amount ₹" + amount + "...");
    try {
        system.pay(receiverUpi, amount);
    } catch (InvalidUPIException | InvalidAmountException | InsufficientBalanceException e) {
        System.out.println("Transaction Failed: " + e.getMessage());
    } finally {
        System.out.print("[Transaction Log Status] ");
        system.checkBalance();
    }
}
}

```



7. Program Execution Output

```

----- Wallet Details -----
User Name       : REVANTH REDDY
Mobile Number  : 9652569999
UPI ID         : krevanthreddy2@upi
Current Balance: ₹1000.0
-----

Adding ₹500 to wallet...
Successfully added ₹500.0 to wallet.

Attempting transaction to merchant@okaxis of amount ₹300.0...
Payment of ₹300.0 to merchant@okaxis was successful!
[Transaction Log Status] Available Balance: ₹1200.0

Attempting transaction to invalidUpiFormat of amount ₹200.0...
Transaction Failed: Invalid UPI ID format: 'invalidUpiFormat'.
[Transaction Log Status] Available Balance: ₹1200.0

Attempting transaction to friend@okicici of amount ₹-50.0...
Transaction Failed: Payment amount must be greater than zero.
[Transaction Log Status] Available Balance: ₹1200.0

Attempting transaction to friend@okicici of amount ₹5000.0...
Transaction Failed: Insufficient balance! Requested: ₹5000.0, Available: ₹1200.0
[Transaction Log Status] Available Balance: ₹1200.0

----- Wallet Details -----

```

```
User Name      : REVANTH REDDY
Mobile Number  : 9652569999
UPI ID        : krevanthreddy2@upi
Current Balance: ₹1200.0
-----
```



8. Key Takeaways & Concepts Demonstrated

- **Encapsulation:** Private members in `Wallet` protect state against direct unauthorized mutation.
- **Abstraction & Interfaces:** `PaymentService` decouples execution from contract definition.
- **Custom Exceptions:** Business logic violations are clearly handled via tailored exceptions rather than generic system errors.
- **Reliable Logging:** The `finally` block ensures consistent post-transaction state reporting.